Docker Interview Guide

Complete preparation guide covering containers, images, Dockerfiles, networking, and real interview Q&A.

1. Containers vs Virtual Machines

This is almost always the opening question in a Docker interview — make sure you can explain it precisely, not just say 'containers are lighter'.

Key differences

- Containers share the host OS kernel via namespaces and cgroups; VMs each run a full guest OS on top of a hypervisor.
- Containers start in milliseconds to seconds; VMs take much longer because they boot an entire OS.
- Containers have a smaller footprint (MBs vs GBs), enabling much higher density per host.
- VMs provide stronger isolation (separate kernel), which matters for strict multi-tenant security boundaries.

2. Docker Architecture

Understand the client-server model that underlies every `docker` command.

Core components

- Docker Daemon (dockerd): background service that builds, runs, and manages containers; listens for API requests.
- Docker Client: the CLI (docker) that talks to the daemon over a REST API.
- Images: read-only templates built in layers, used to create containers.
- Containers: runnable, writable instances of images.
- Registry: stores and distributes images (Docker Hub, Amazon ECR, GitHub Container Registry, private registries).

3. Dockerfile Instructions

Interviewers frequently ask you to explain or write a Dockerfile, and to justify instruction ordering for build efficiency.

Key instructions

- FROM: sets the base image for subsequent instructions.
- RUN: executes a command at build time and commits the result as a new layer.
- CMD vs ENTRYPOINT: CMD provides default arguments that can be overridden at `docker run`; ENTRYPOINT fixes the executable that always runs, with CMD supplying default arguments to it.
- COPY vs ADD: COPY simply copies files/directories from build context into the image; ADD does the same but also supports remote URLs and automatic extraction of local tar archives — COPY is generally preferred for clarity.
- EXPOSE: documents which port the container listens on (does not actually publish it).
- WORKDIR: sets the working directory for subsequent instructions.
- ENV / ARG: ENV sets runtime environment variables; ARG defines build-time variables only.

Sample multi-stage Dockerfile

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn package -DskipTests

FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=build /app/target/app.jar .
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

4. Docker Compose & Networking

Multi-container applications are the norm in real projects, so expect at least one practical Compose question.

Docker Compose

- Defines and runs multi-container applications declaratively in a docker-compose.yml file.
- Handles service dependencies, shared networks, and named volumes automatically.
- `docker compose up -d` starts everything in the background; `docker compose down` tears it down.

Networking & Volumes

- Bridge network (default): isolated network for containers on a single host, with internal DNS resolution by service/container name.
- Host network: container shares the host's network stack directly (no isolation, best performance).
- Volumes: managed by Docker, ideal for persisting database data outside the container lifecycle. Bind mounts map a host path directly into the container.

5. Real Interview Q&A

Common technical questions evaluating practical knowledge and architectural understanding.

Q: Why are Docker images layered, and why does that matter for performance?

A: Each instruction in a Dockerfile (except a few metadata-only ones) creates a new, cached, read-only layer. Layers are shared and reused across images with a common base, saving disk space and speeding up builds and pulls. Because of caching, ordering instructions so that rarely-changing steps (like installing dependencies) come before frequently-changing steps (like copying source code) dramatically speeds up rebuilds — Docker only rebuilds from the first changed layer onward.

Q: What is the difference between CMD and ENTRYPOINT?

A: ENTRYPOINT defines the fixed command that always executes when the container starts, while CMD supplies default arguments to it (or, if ENTRYPOINT isn't set, defines the whole default command). Arguments passed to `docker run image ` override CMD but are appended to ENTRYPOINT. A common pattern is ENTRYPOINT ["java","-jar","app.jar"] with CMD providing default JVM flags that a user can override.

Q: How do you reduce Docker image size?

A: Use a minimal base image (alpine or distroless), use multi-stage builds so build tools and intermediate artifacts don't ship in the final image, combine RUN commands to reduce layers, clean up package manager caches in the same layer they're created, and add a .dockerignore file to keep unnecessary files out of the build context.

Q: Explain multi-stage builds and why they're useful.

A: A multi-stage build uses multiple FROM statements in one Dockerfile, where each stage can copy artifacts from a previous stage using COPY --from. This lets you compile or build an application in a heavyweight stage (with compilers, SDKs, build tools) and then copy only the final artifact into a lightweight runtime image, dramatically shrinking the final image size and reducing its attack surface.

Q: How does container networking work by default, and how do containers discover each other?

A: By default, Docker creates a bridge network per host; containers attached to the same user-defined bridge network can resolve each other by container/service name via Docker's built-in DNS. Containers on different networks are isolated unless explicitly connected. In Docker Compose, all services in a compose file share a default network automatically.

Q: What's the difference between a volume and a bind mount?

A: A named volume is managed entirely by Docker and stored in a Docker-controlled location on the host — portable, easy to back up, and the recommended way to persist data like database files. A bind mount maps an arbitrary path on the host filesystem directly into the container, which is more flexible for local development (e.g., live-reloading source code) but tightly couples the container to the host's filesystem layout.

Q: How would you debug a container that keeps crashing on startup?

A: Start with `docker logs ` to see stdout/stderr from the failed process, and `docker ps -a` to check its exit code. `docker inspect` reveals configuration and recent state. Running the image interactively with an overridden entrypoint (`docker run -it --entrypoint sh image`) lets you poke around the filesystem and manually run the startup command to see the real error.

Q: What is the difference between COPY and ADD?

A: COPY does a straightforward copy of files or directories from the build context into the image. ADD does the same, but additionally supports fetching remote URLs and automatically extracting local compressed archives (like .tar.gz) into the destination. Because ADD's extra behavior can be surprising, COPY is the recommended default unless you specifically need ADD's extraction feature.

Q: How do health checks work in Docker, and why use them?

A: A HEALTHCHECK instruction (or Compose healthcheck block) defines a command Docker runs periodically inside the container to determine if the application is actually working, not just that the process is running. Docker reports the container status as healthy, unhealthy, or starting, which orchestrators (like Swarm or Compose with depends_on: condition: service_healthy) use to make restart or routing decisions.

Q: What happens to data inside a container when it stops or is removed?

A: A container's writable layer is ephemeral — data written there is lost when the container is removed (though it survives a stop/start cycle). Any data that must persist beyond the container's lifecycle should be stored in a named volume or bind mount, which exists independently of the container.

Pro Tips for the Interview

- Be ready to write a real Dockerfile from scratch for a simple app — interviewers love live-coding this.
- Always mention multi-stage builds when asked about image size or production best practices.
- Know exactly why CMD vs ENTRYPOINT matters, with an example — it's a very commonly asked distinction.
- Mention .dockerignore when discussing build context and image size; it's an easy point that's often forgotten.